



Cyber Security

Project Report

Project 1: Password Strength Checker

Track: Junior Analyst — Defensive Logic
Batch: 2026
Prepared for: DecodeLabs Internship Program

Table of Contents

1. Introduction & Project Objective.....	2
2. Requirements & Key Skills.....	2
3. Why Password Strength Matters.....	3
4. Design Logic — The IPO Model.....	3
5. Source Code Implementation.....	4
6. Testing & Execution Output.....	5
7. Security Considerations & Future Improvements.....	6
8. Conclusion.....	7

1. Introduction & Project Objective

This report documents the design, implementation, and testing of Project 1 of the DecodeLabs Cyber Security Industrial Training Kit: a Password Strength Checker. The project marks the entry point into the “defensive logic” track, where the focus is not on offensive hacking techniques but on the fundamentals of secure input validation, string handling, and conditional decision-making.

Before an organization can defend a large network, its analysts must first understand how a single credential can be evaluated for risk. This program simulates that evaluation: it accepts a password typed by the user, inspects its structure, and classifies it as Weak, Medium, or Strong based on a defined security policy.

Objective

To build a Python program that checks whether a given password is weak, medium, or strong by examining its length and the variety of character types it contains, and to display the result to the user.

Scope

The scope of this milestone is limited to rule-based (heuristic) password classification using pure string handling and conditional logic — no external libraries, hashing, or encryption are involved at this stage. Those concepts are reserved for Project 2 of the training kit.

2. Requirements & Key Skills

Key Requirements

- Check the length of the password entered by the user.
- Check for the use of uppercase letters.
- Check for the use of numeric digits.
- Check for the use of special symbols.
- Classify and display the final password strength result (Weak / Medium / Strong).

Key Skills Demonstrated

- String handling — iterating over each character of user input.
- Conditional logic — nested and compound if / elif / else structures.
- Boolean flag tracking — using state variables to accumulate character-type evidence.
- Security basics — translating an informal policy (“use a mix of characters”) into enforceable code.

Qualification Criteria

Per the training guidelines, completion of Project 1 is mandatory and unlocks subsequent projects for the following week. All submitted work is verified for quality before an intern is credited with the milestone badge.

3. Why Password Strength Matters

Weak credentials remain one of the most common entry points for attackers. According to the Verizon Data Breach Investigations Report (DBIR), a large majority of hacking-related breaches leverage weak or stolen passwords, which makes automated strength validation a foundational control in any authentication system.

A significant share of enterprise authentication traffic is generated by automated credential-stuffing and injection attempts, and the average cost of a data breach is measured in millions of dollars. A simple, well-designed strength check at the point of account creation meaningfully reduces this exposure by rejecting predictable, low-entropy passwords before they are ever stored.

The Zero Point: Defining the Policy

- Length verification: fewer than 8 characters is treated as an immediate fail, since short passwords carry an exponentially higher brute-force risk.
- Pattern recognition: the policy mandates a mix of uppercase letters, digits, and symbols.
- The Unicode curveball: restricting input to the 95-character ASCII set caps the theoretical search space; allowing the full Unicode range expands it to well over 140,000 possible characters, dramatically increasing entropy.

4. Design Logic — The IPO Model

The program follows a classic Input → Process → Output structure. The password string is treated as the raw input; a single linear pass over its characters (an $O(n)$ scan) builds up evidence of which character types are present; and the accumulated evidence is fed into a decision tree that produces one of three risk classifications.

Computational Efficiency

A professional implementation favors Python's built-in, C-optimized helpers over verbose manual loops. For example, rather than writing an explicit for-loop with a break statement to detect a digit, an idiomatic implementation can use a generator expression such as `any(char.isdigit() for char in password)` — the same idea applies to `isupper()` and `isalnum()` checks used in this project.

The Decision Tree

- Strong Password: length ≥ 8 AND contains an uppercase letter AND a digit AND a symbol.
- Medium Password: length ≥ 6 AND (contains an uppercase letter OR a digit).
- Weak Password: any password that does not satisfy the Medium or Strong conditions above.

This constraint — that validation time must grow linearly and never exponentially with input size — is what keeps the checker responsive even for very long passwords, since every character is visited exactly once.

5. Source Code Implementation

The complete, final implementation is shown below. It uses three Boolean flags (`has_upper`, `has_number`, `has_symbol`) that are updated during a single pass through the password string, followed by a compound conditional block that maps the collected flags and the password length to a strength label.

```
password_checker.py
password = input("Enter your password: ")

length = len(password)

has_upper = False
has_number = False
has_symbol = False

for char in password:
    if char.isupper():
        has_upper = True
    if char.isdigit():
        has_number = True
    if not char.isalnum():
        has_symbol = True

if length >= 8 and has_upper and has_number and has_symbol:
    print("Strong Password")

elif length >= 6 and (has_upper or has_number):
    print("Medium Password")

else:
    print("Weak Password")
```

Code Walkthrough

- Lines 1–3: The password is captured via `input()` and its length is stored once in a variable to avoid recomputing `len()` repeatedly.
- Lines 5–7: Three Boolean flags are initialised to `False` before the scan begins.
- Lines 9–15: A single for-loop visits every character exactly once, updating the relevant flag whenever an uppercase letter, digit, or non-alphanumeric symbol is found.
- Lines 17–24: A compound `if / elif / else` block reads the length and the three flags to print the final classification.

6. Testing & Execution Output

The script was executed three times from the terminal inside Visual Studio Code, using progressively stronger inputs to validate each branch of the decision tree. The screenshot below captures the editor, the final version of the source file, and the terminal transcript of all three runs.

Test Case Summary

Input	Composition	Result
hello	5 chars, no upper, no digit, no symbol	Weak Password
hello@123	9 chars, digit + symbol, no upper	Medium Password
Hello@123	9 chars, upper + digit + symbol	Strong Password

Terminal Evidence

The screenshot shows the Visual Studio Code editor with the `password_checker.py` file open. The file contains the following Python code:

```

1 password = input("Enter your password: ")
2
3 length = len(password)
4
5 has_upper = False
6 has_number = False
7 has_symbol = False
8
9 > for char in password: ...
18
19 if length >= 8 and has_upper and has_number and has_symbol:
20     print("Strong Password")
21
22 elif length >= 6 and (has_upper or has_number):
23     print("Medium Password")
24
25 else:
26     print("Weak Password")

```

The terminal output shows three test runs of the script:

```

Enter your password: hello
Weak Password
PS C:\Users\PMILS\Desktop\password_checker> python password_checker.py
Enter your password: hello@123
Medium Password
PS C:\Users\PMILS\Desktop\password_checker> python password_checker.py
Enter your password: Hello@123
Strong Password
PS C:\Users\PMILS\Desktop\password_checker>

```

Figure 1 — VS Code editor and terminal output showing all three test runs of `password_checker.py`

7. Security Considerations & Future Improvements

While this checker satisfies the Project 1 brief, the broader training kit highlights several advanced considerations that would apply once this logic is connected to a real authentication system:

- Data-in-memory exposure: Python strings are immutable, so a plaintext password cannot be overwritten in place and may linger in RAM until garbage collection — a real system should minimise how long the raw password is held in memory.
- Timing attacks: naive character-by-character comparisons can leak information through execution-time differences; constant-time comparison functions (e.g. `hmac.compare_digest`) should be used wherever secrets are compared, not just when checking strength.
- Validation before encryption: strength checking must always happen before hashing, since a weak password cannot be made strong by hashing it afterward — this “gatekeeper rule” is the bridge to Project 2 (Hashing & Encryption).
- Leaked-password lists: a production-grade checker would also reject passwords found in known breach corpora, not just enforce character variety.
- Suggested extension: adding a check against a small list of common passwords, and reporting **which** rule failed rather than only the final label, would make the tool more useful for end users.

8. Conclusion

This project delivered a working password strength checker that satisfies every requirement set out in the DecodeLabs Project 1 brief: it measures password length, detects the presence of uppercase letters, digits, and symbols through a single linear scan, and reports a clear Weak / Medium / Strong classification. Testing against three representative inputs confirmed that all three branches of the decision tree behave as designed.

Beyond the code itself, the exercise reinforced the underlying security mindset behind authentication systems: that robust protection begins with rigorous input validation, long before any cryptography is involved. The best way to build on this milestone is hands-on iteration — experimenting with additional rules, testing edge cases, and treating every logic error encountered along the way as a learning opportunity.

This report, together with the accompanying source file, forms the first entry in a growing portfolio intended to demonstrate a practical, security-first engineering mindset for future employers.
